

OPERATING SYSTEMS

Pintos: Report Project #2

Group #6

Davide Frova, Costanza Rodriguez Gavazzi

Fall Semester, March 2, 2024

1 FILES CHANGED

- `/pintos/threads/sleepy_thread.h` (*added*)
- `/pintos/devices/timer.c` (*modified*)

2 CHANGES

`/pintos/thread/sleepy_thread.h`

- Created this header file declaring **struct sleepy_thread** in order to create a list with **lib/kernel/list.h**. The struct is built in order to accomodate the requirements of **list.h**: it contains the mandatory **struct list_elem elem**, a **struct thread * thread_p** that points to the current thread which needs to be put to sleep, a **int64_t awake_time** (which represents when a thread needs to be woken up in terms of absolute ticks).

`/pintos/devices/timer.c`

- (*added*) Includes:
 - `threads/sleepy_thread.h`
 - `threads/tests.h`
- (*added*): **static struct list sleepy_threads_list**; This static global variable is needed to store the threads that are currently blocked and are waiting to be unblocked after some time ticks.
- `void timer_init(void)`: (*modified*): We initialize the sleepy list here because we wanted it to be done as soon as possible and this is the first method that runs when using `timer.c`

- `bool compare_awake_time(const struct list_elem *t1_, const struct list_elem *t2_, void *aux UNUSED);` (*added*): This method is needed to use the `list_insert_ordered` method in `list.c`. It takes two sleepy threads and checks whether `t1` needs to be awakened before `t2`.
- `void timer_sleep(int64_t ticks);` (*modified*):
 - Disabled interrupts when entering critical section;
 - For the current thread, it computes its `awake_time` by summing the current tick time and the desired sleep duration that is passed as parameter `ticks`;
 - It inserts the thread in the sleepy threads list
 - When exiting critical section we re-enable interrupts
- `static void timer_interrupt(struct intr_frame args UNUSED)` (*modified*):
 - Disabled interrupts when entering critical section;
 - Iterating through the `sleepy_threads_list` field in `timer.c`: we check if there are threads that need to be unblocked by accessing a dedicated field of the list element (`awake_time`) and do it.
 - When exiting critical section we re-enable interrupts